P E S U N I V E R S I T Y
Department of Computer Science and Engineering
Blockchain | UE23CS342BA5

On-Chain Leaderboard
for Coding Contest

A Transparent, Tamper-Proof Contest Judging System on Ethereum

TEAM MEMBERS	
Kathit Joshi PES2UG23CS264	**Kavyansh Jain** PES2UG23CS268
Kartik Bhat Sumbly PES2UG23CS258	

Academic Year 2025 – 2026

1. Problem Statement

Coding contest results are traditionally managed by a central organizer or platform. This centralised approach introduces several critical vulnerabilities that undermine the integrity of competitive evaluation.

Score Manipulation Scores can be silently altered after the fact with no audit trail to detect changes.	**No Transparency** Participants have no way to verify how scores were assigned or which judge evaluated them.
Bias and Favouritism Without assignment enforcement, judges can be influenced to score participants outside their purview.	**Disputed Results** No immutable record means results can be disputed with no way to definitively resolve the dispute.

Our solution addresses all of these problems by deploying the entire judging process on the Ethereum blockchain. Every registration, score submission, and finalization is recorded as an immutable on-chain transaction — visible to all, alterable by none.

2. Solution Overview

We built a transparent contest judging system with the following guarantees:

- The organizer deploys the contract and registers participants and judges
- Each judge is assigned specific participants — cross-assignment scoring is rejected by the contract
- Judges submit scores across three criteria: Problem Solving (40 pts), Code Quality (35 pts), and Efficiency (25 pts)
- The contract automatically calculates and maintains a weighted aggregate score per participant
- Once a judge submits a score it cannot be edited or overwritten by anyone
- Finalization by the organizer is only possible after all judges complete all their assigned evaluations
- After finalization, no score changes are permitted by any wallet — including the organizer
- The frontend displays live judging progress, final leaderboard with podium, and full per-judge score breakdowns

Real-World Applications

University Hackathons	Multiple faculty judges, fully transparent scoring, no post-event manipulation possible
National Coding Olympiads	Government-auditable on-chain results with complete evaluation history
Scholarship Committees	Bias-proof merit evaluation with full immutable audit trail per evaluator

3. Technologies Used

Tool / Technology	Category	Role in Project
Solidity 0.8.19	Smart Contract Language	Core contract logic — state machine, access control, scoring, events, custom errors
Remix IDE	Development Environment	Contract development, compilation, deployment to Ganache via Injected Provider
Ganache	Local Blockchain	Local Ethereum test network — provides 10 pre-funded accounts, instant mining
MetaMask	Wallet / Provider	Browser extension acting as the Web3 provider — signs and sends all transactions
Web3.js v1.10	JavaScript Library	Frontend library for connecting to Ethereum — calls contract functions from HTML
HTML / CSS / JS	Frontend	Single-file dApp — no framework, runs via Python HTTP server on localhost
Python HTTP Server	Dev Server	Serves the HTML file over http:// so MetaMask can inject window.ethereum

4. Solidity Concepts Used

The smart contract incorporates the following Solidity concepts covered in the course:

`enum`	ContestState { REGISTRATION, ACTIVE, FINALIZED } — enforces lifecycle transitions
`struct`	Score, Judge, Participant — organise complex data per entity
`mapping`	participants, judges, scores, isAssigned — O(1) lookups by address
`modifier`	onlyOrganizer, onlyJudge, inRegistration, inActive, notFinalized
`custom errors`	NotOrganizer, ParticipantNotAssigned, ScoreAlreadySubmitted etc. — gas efficient vs require strings
`events`	8 events emitted on every state change — full on-chain audit trail
`immutable`	organizer address set once in constructor, cannot be changed
`constructor`	Initialises contest name, sets organizer, emits ContestCreated event
`memory vs storage`	Function params use memory; state variables use storage correctly
`view functions`	getLeaderboard, getScore, isReadyToFinalize — read-only, no gas cost
`access control`	Role-based enforcement: only organizer registers, only assigned judges score
`state machine`	Three-state lifecycle with enforced one-way transitions

5. Code Snippets

Key excerpts from ContestJudging.sol demonstrating the core Solidity concepts:

5.1 Contract Header, Enums and Structs

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.19;

contract ContestJudging {

    // Lifecycle states of the contest
    enum ContestState {
        REGISTRATION,   // Organizer is adding participants/judges
        ACTIVE,         // Judging in progress
        FINALIZED       // Results locked permanently
    }

    struct Score {
        uint8   problemSolving;  // Max 40 points
        uint8   codeQuality;     // Max 35 points
        uint8   efficiency;      // Max 25 points
        uint16  aggregate;       // Weighted total (out of 100)
        bool    submitted;       // Guard against re-submission
        uint256 timestamp;       // Block timestamp of submission
    }

    struct Participant {
        address wallet;
        string  name;
        bool    registered;
        uint256 totalAggregate;
        uint256 judgeCount;
        uint256 finalScore;
        uint8   rank;
    }
}
```

5.2 State Variables, Custom Errors and Events

```
    address public immutable organizer;
    ContestState public contestState;
    uint256 public constant MAX_PROBLEM_SOLVING = 40;
    uint256 public constant MAX_CODE_QUALITY     = 35;
    uint256 public constant MAX_EFFICIENCY       = 25;

    mapping(address => Participant) public participants;
    mapping(address => Judge)       public judges;
    mapping(address => mapping(address => Score)) public scores;
    mapping(address => mapping(address => bool))  public isAssigned;

    // Gas-efficient custom errors
    error NotOrganizer();
    error ParticipantNotAssigned(address judge, address participant);
    error ScoreAlreadySubmitted(address judge, address participant);
    error JudgesHaveIncompleteSubmissions();
    error AlreadyFinalized();

    // Full audit trail events
    event ScoreSubmitted(
        address indexed judge,
        address indexed participant,
```

```
        uint8 problemSolving, uint8 codeQuality, uint8 efficiency,
        uint16 aggregate, uint256 timestamp
    );
    event ContestFinalized(uint256 timestamp, address topRanked);
```

5.3 Modifiers and Access Control

```
    modifier onlyOrganizer() {
        if (msg.sender != organizer) revert NotOrganizer();
        _;
    }

    modifier onlyJudge() {
        if (!judges[msg.sender].registered) revert NotAJudge();
        _;
    }

    modifier inRegistration() {
        if (contestState != ContestState.REGISTRATION)
            revert NotInRegistration();
        _;
    }

    modifier inActive() {
        if (contestState != ContestState.ACTIVE)
            revert NotInActiveState();
        _;
    }
```

5.4 submitScore — Core Judging Function

```
    function submitScore(
        address _participant,
        uint8   _problemSolving,
        uint8   _codeQuality,
        uint8   _efficiency
    ) external onlyJudge inActive {

        // Reject if participant not assigned to this judge
        if (!isAssigned[msg.sender][_participant])
            revert ParticipantNotAssigned(msg.sender, _participant);

        // Reject re-submission — immutable once submitted
        if (scores[msg.sender][_participant].submitted)
            revert ScoreAlreadySubmitted(msg.sender, _participant);

        // Validate individual scores against maximums
        if (_problemSolving > MAX_PROBLEM_SOLVING)
            revert InvalidScore('problemSolving', _problemSolving,
MAX_PROBLEM_SOLVING);

        uint16 aggregate = uint16(_problemSolving)
                         + uint16(_codeQuality)
                         + uint16(_efficiency);

        scores[msg.sender][_participant] = Score({
            problemSolving: _problemSolving,
            codeQuality:    _codeQuality,
            efficiency:     _efficiency,
            aggregate:      aggregate,
            submitted:      true,
            timestamp:      block.timestamp
```

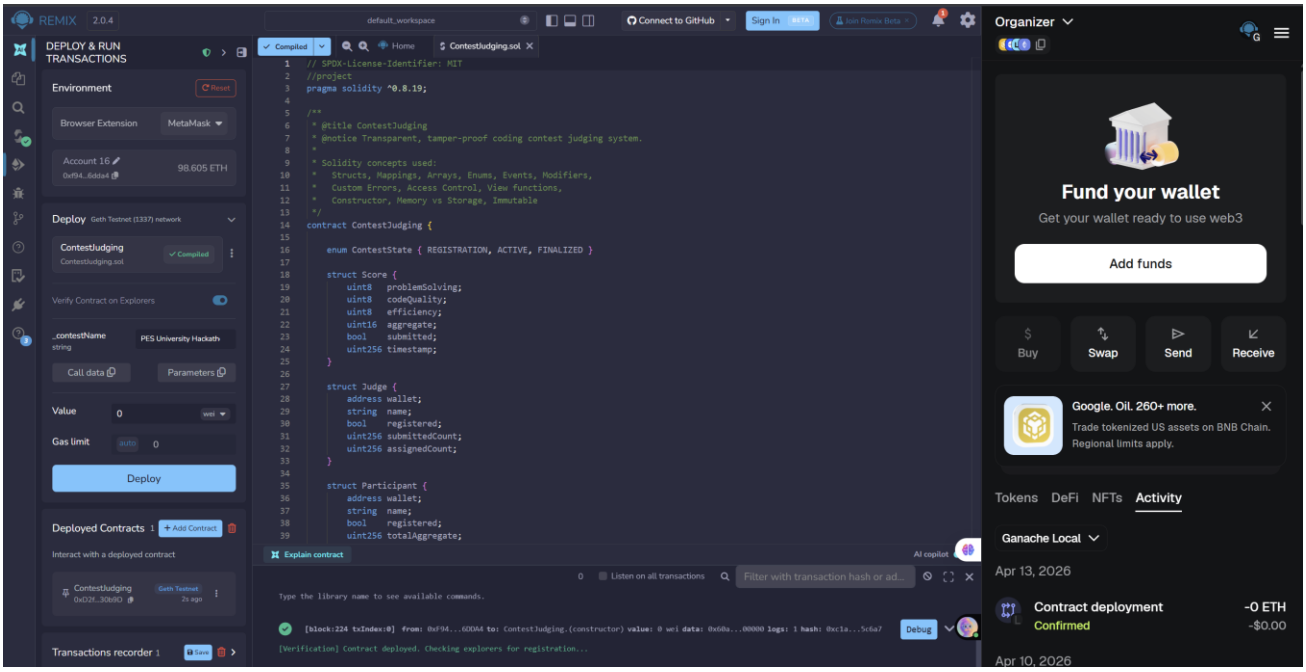
```
        });

        participants[_participant].totalAggregate += aggregate;
        participants[_participant].judgeCount++;
        judges[msg.sender].submittedCount++;

        emit ScoreSubmitted(msg.sender, _participant,
            _problemSolving, _codeQuality, _efficiency,
            aggregate, block.timestamp);
    }
```

5.5 finalizeContest and Internal Sorting

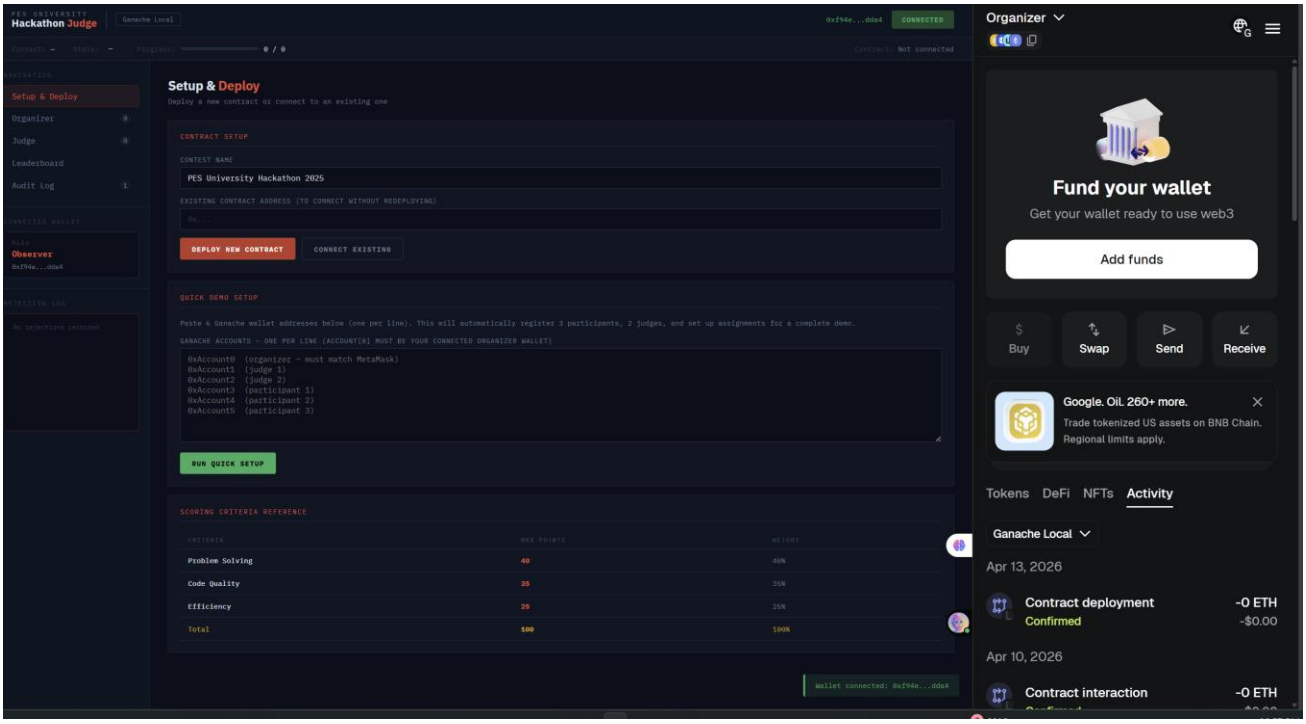
```
    function finalizeContest() external onlyOrganizer inActive {
        // All judges must have completed their assignments
        for (uint256 i = 0; i < judgeList.length; i++) {
            address judge = judgeList[i];
            if (judges[judge].submittedCount < judges[judge].assignedCount)
                revert JudgesHaveIncompleteSubmissions();
        }

        // Calculate final average score per participant
        for (uint256 i = 0; i < participantList.length; i++) {
            address p = participantList[i];
            if (participants[p].judgeCount > 0)
                participants[p].finalScore =
                    participants[p].totalAggregate / participants[p].judgeCount;
            rankedParticipants.push(p);
        }

        _sortLeaderboard();   // Bubble sort descending by finalScore

        for (uint256 i = 0; i < rankedParticipants.length; i++) {
            participants[rankedParticipants[i]].rank = uint8(i + 1);
            emit RankAssigned(rankedParticipants[i], uint8(i+1),
                participants[rankedParticipants[i]].finalScore);
        }

        contestState = ContestState.FINALIZED;
        emit ContestFinalized(block.timestamp, rankedParticipants[0]);
    }
```

6. Output / Screenshots

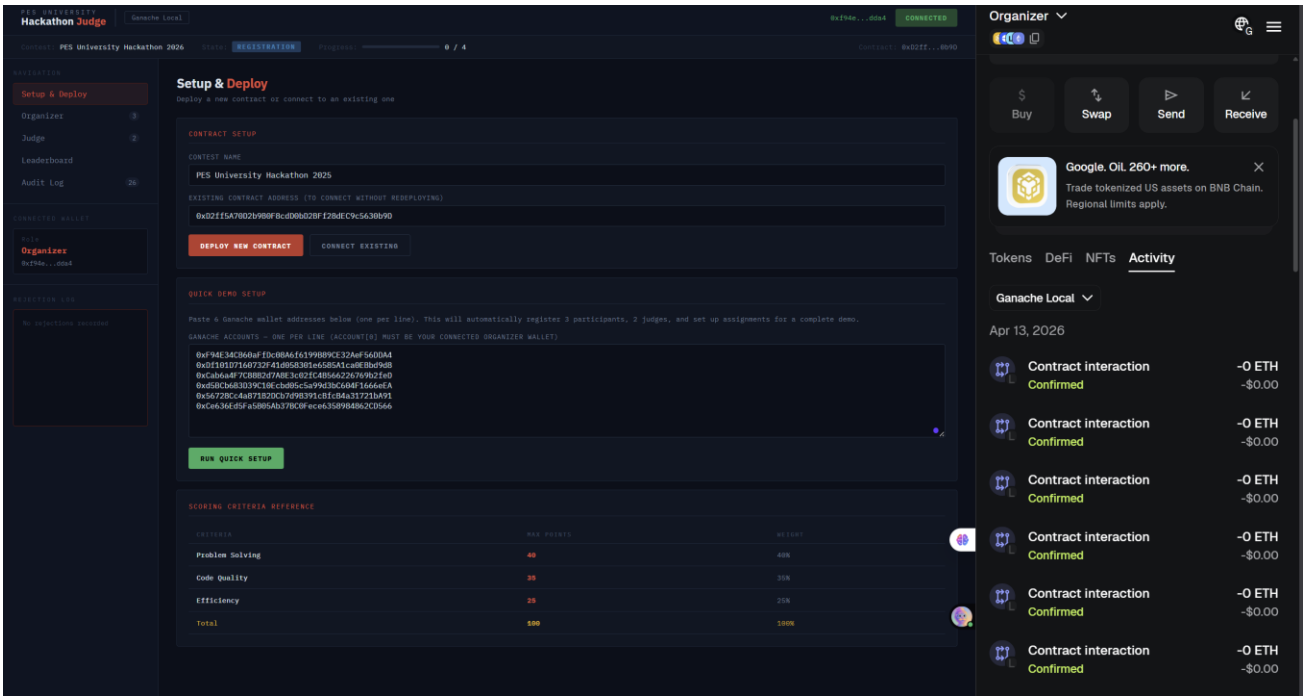
(i) Contract deployment



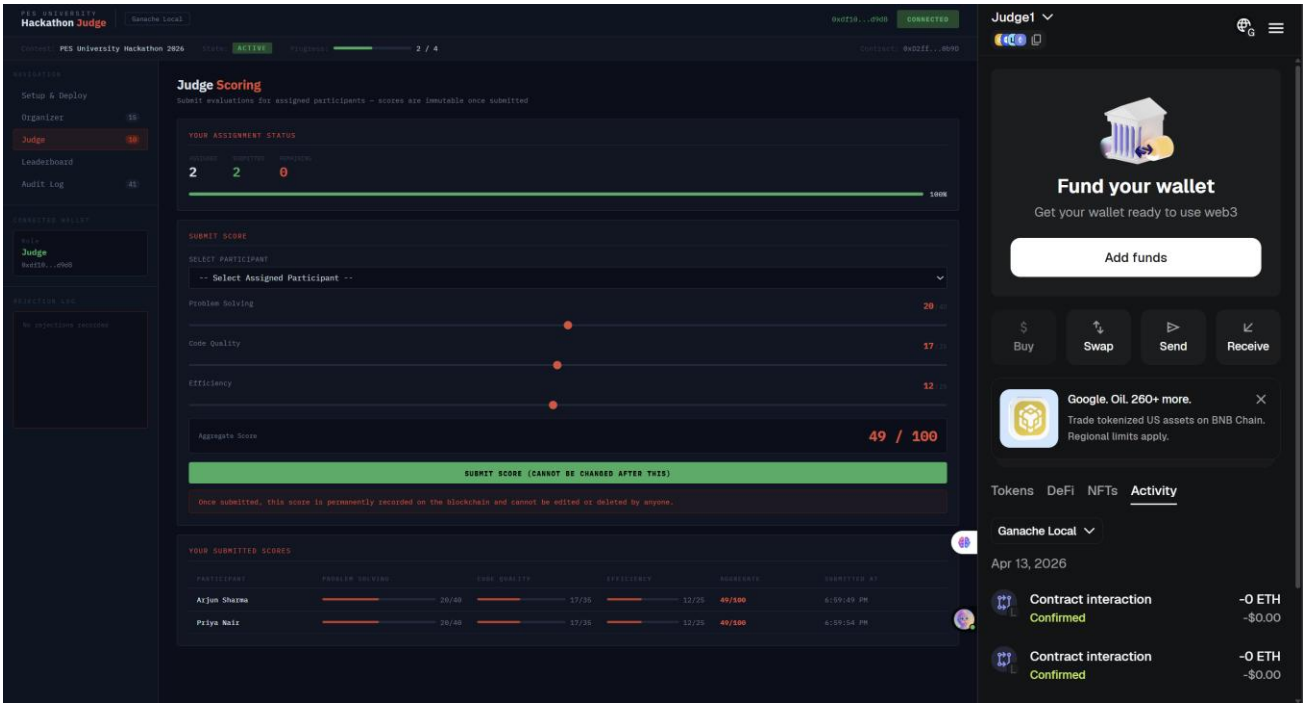
(ii) UI homepage

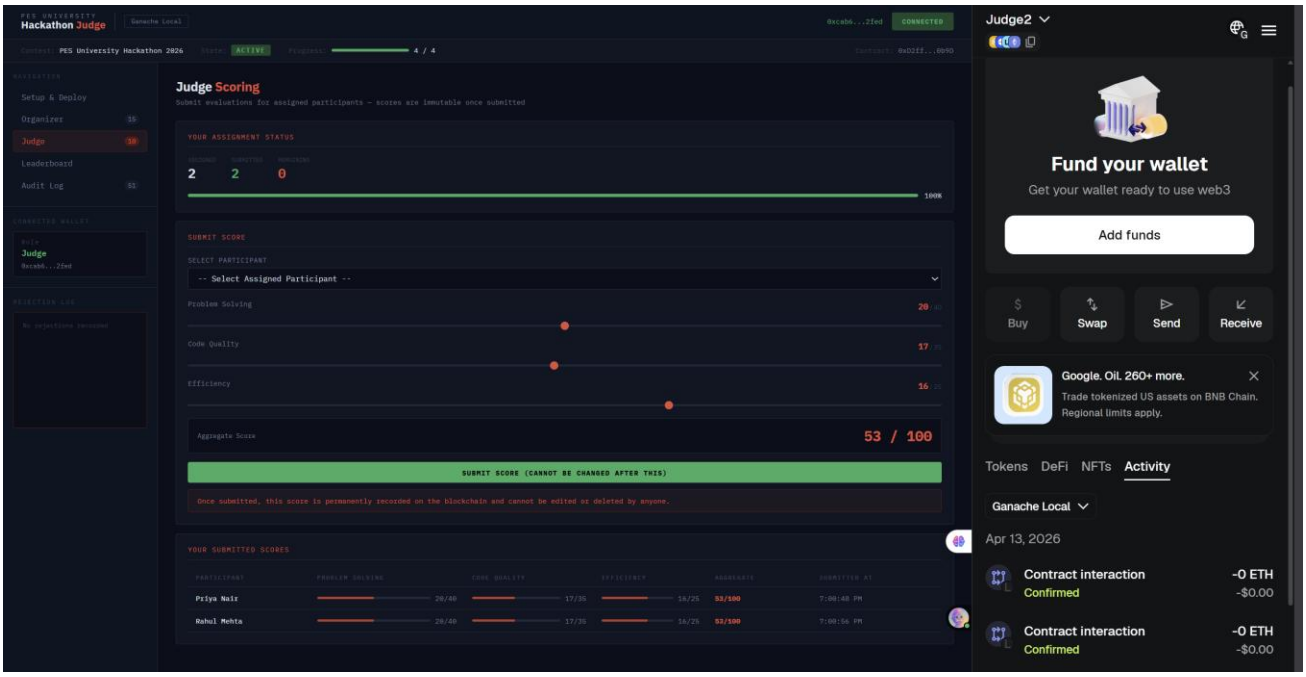


(iii) Connecting to contract on remix and running quick setup

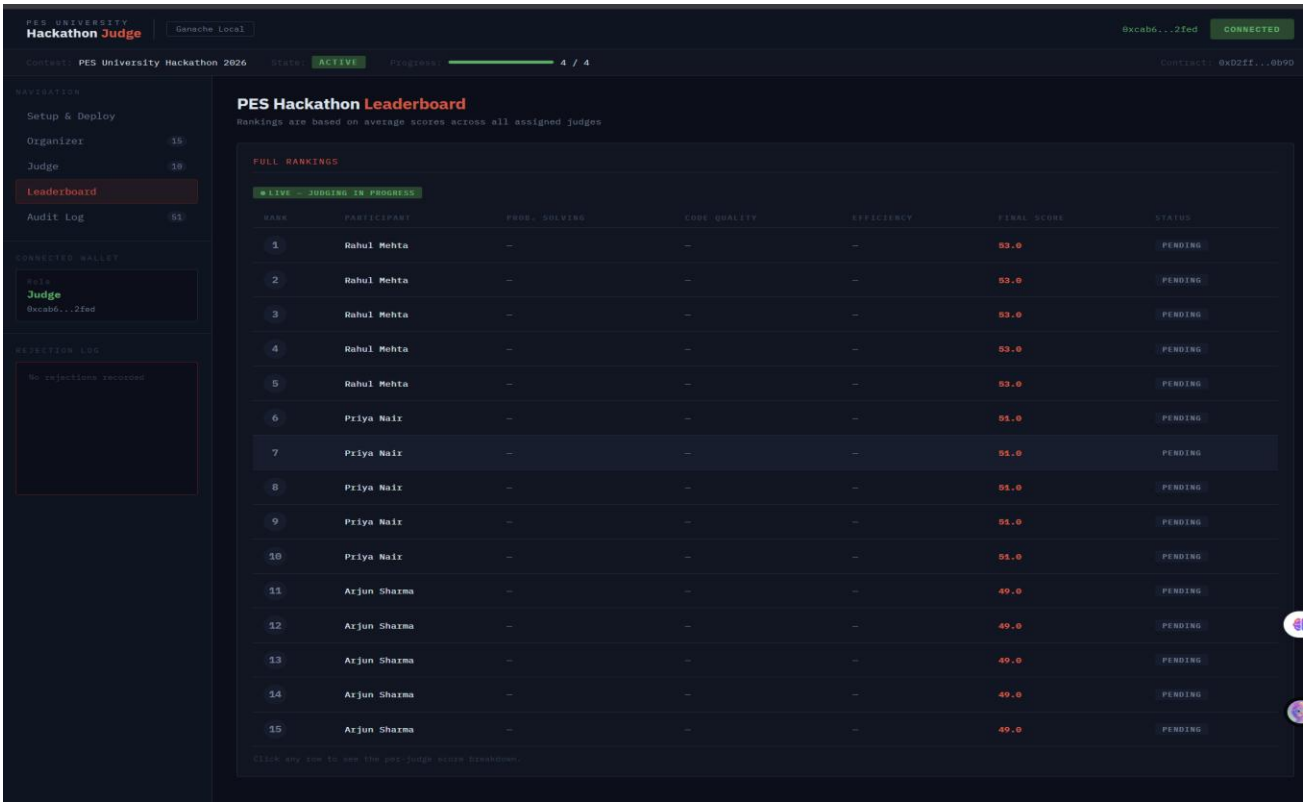


(iv) Starting Judging phase and assigning scores to participants as judge (1st is judge1 and 2nd judge 2)

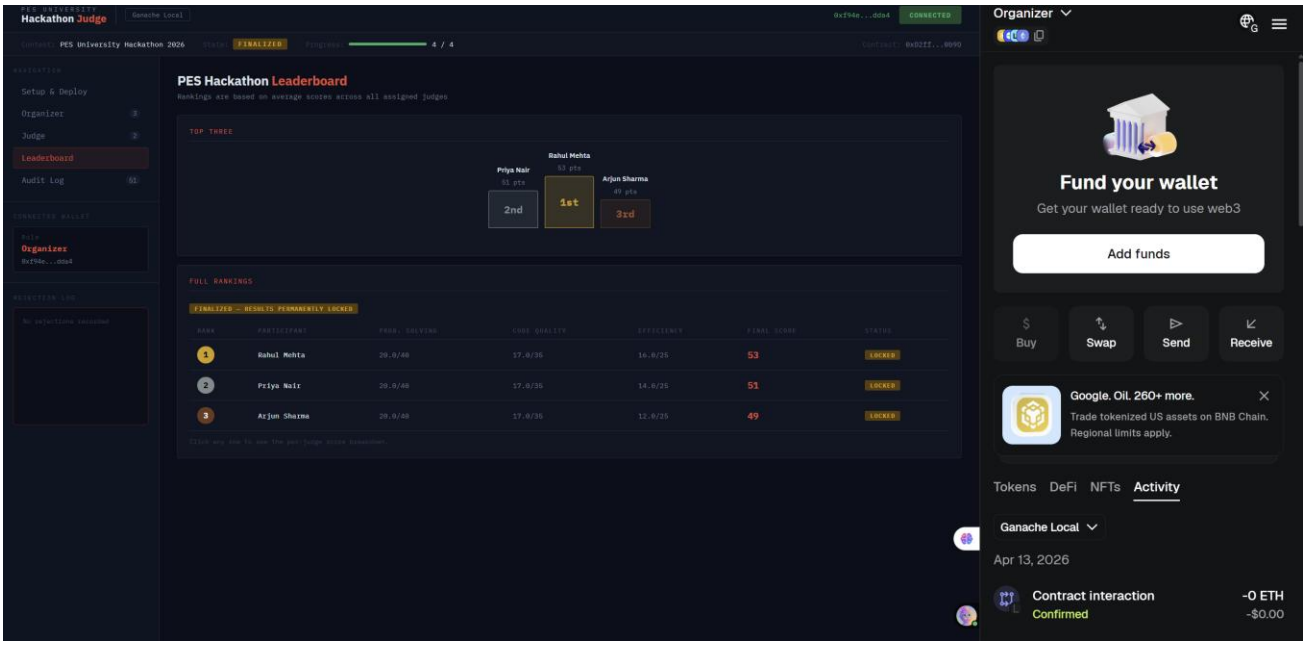
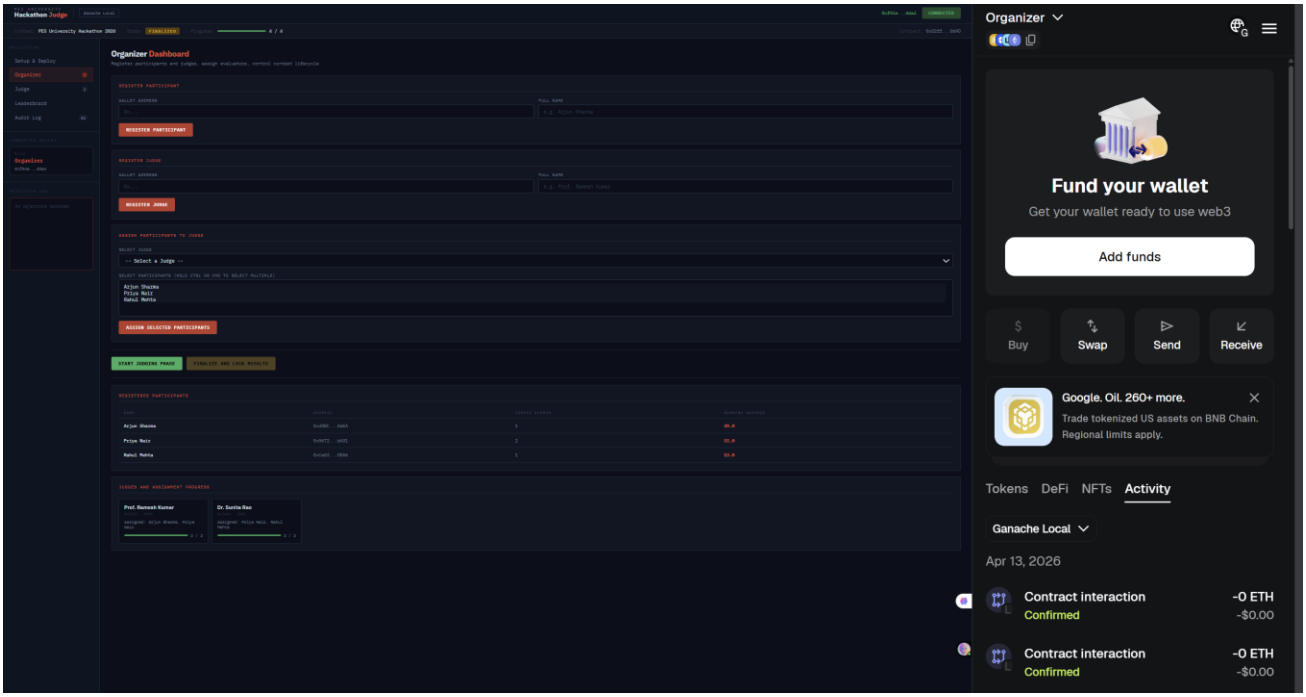




(v) Leaderboard before finalizing and locking it (duplicated entries)



(vi) Leaderboard after finalizing and locking it (avg taken for Priya as she's judged by two judges)



(vii) Organizer dashboard after finalizing and locking leaderboard